DOCKER VOLUMES

Complete Guide

Data Persistence & Sharing

Created: March 29, 2026
DevOps Learning Guide

■ TABLE OF CONTENTS

1. Why Volumes? (Data Replication)

2. Types of Volumes

3. Container-to-Container Sharing

4. Images vs Volumes

5. Named Volumes vs Bind Mounts

6. Creating Volumes

7. Real-World Examples

8. Docker Compose + Volumes

9. Best Practices

1■■ WHY VOLUMES? DATA REPLICATION

What is a Volume?

A **volume** is a shared folder on your host machine that persists data independently of container lifecycle. Multiple containers can mount the same volume to share and sync data automatically.

Key Points:

■ **Data Persistence:** Data survives container restart/deletion

■ **Container-to-Container Sharing:** Multiple containers access same data

■ **Automatic Sync:** Changes in one container reflect in others instantly

■ **Independent:** Volumes are separate from container filesystem

2■■ TYPES OF VOLUMES

Type A: Named Volumes

```
docker volume create FLM-Volume
docker run -it --name cont1 -v FLM-Volume:/app/data ubuntu
```

• **Managed by Docker**
• **Hidden location:** /var/snap/docker/common/var-lib-docker/volumes/
• **Best for:** Production

Type B: Bind Mounts

```
mkdir /root/myapp
docker run -it --name cont1 -v /root/myapp:/data ubuntu
```

• **You control location**
• **Visible in your folder**
• **Best for:** Development

Type C: Using $(pwd)

```
docker run -v $(pwd):/app ubuntu
```

• **Mounts current directory**
• **Convenient for projects**

3■■ CONTAINER-TO-CONTAINER SHARING

Using --volumes-from

```
docker run -it --name cont1 -v FLM-Volume:/app/data ubuntu
docker run -it --name cont2 --volumes-from cont1 ubuntu
```

What happens?

• Container 2 inherits ALL volumes from Container 1
• Both containers mount to same location
• Changes in cont2 → visible in cont1 instantly ■
• Changes in cont1 → visible in cont2 instantly ■

Real-Time Sync Example:

```
# Inside Container 1
echo 'original' > /app/data/file.txt

# Exit and go to Container 2
cat /app/data/file.txt
# Output: original ■

echo 'updated by cont2' > /app/data/file.txt

# Back in Container 1
cat /app/data/file.txt
# Output: updated by cont2 ■
```

4■■ IMAGES VS VOLUMES (CRITICAL!)

Key Understanding:

Images capture container filesystem.
Volumes are independent storage (NOT in images).

When you commit a container to image, only filesystem files are included!

Aspect	Image	Volume
Filesystem Files	■ Included	■ NOT included
Data Persistence	■ Lost on delete	■ Survives deletion
Portability	■ Portable	■ Host-dependent
Container Delete	■ Files lost	■ Data intact

Example Workflow:

```
Container 1:
■■■ Filesystem: 10 files → saved in IMAGE ■
■■■ Volume: 3 files → NOT in image ■

docker commit cont1 my_image

Container 3 (from my_image):
■■■ Filesystem: 10 files ■ (from image)
■■■ Volume: EMPTY ■ (unless mounted)
```

■■ Important: Always use volumes for important data, not container filesystem!

5■■ NAMED VOLUMES VS BIND MOUNTS

Feature	Named Volume	Bind Mount
Where Stored	Docker manages	You choose
Control	Docker	Full (you)
Sync	Automatic	Automatic
Visibility	Hidden in Docker	Your folder
Best For	Production	Development
Example	-v mydata:/data	-v /root/myapp:/data

When to Use:

Named Volumes:
■ Production database (PostgreSQL, MySQL)
■ Important data
■ Docker manages everything

Bind Mounts:
■ Development (editing code)
■ Project source code
■ Want to see files in your folder

6■■ CREATING VOLUMES

Method 1: Via -v flag

```
docker run -it --name cont1 -v mydata:/app/data ubuntu
```

Method 2: Via --mount flag

```
docker run -it --name cont1 --mount source=mydata,destination=/app/data ubuntu
```

Method 3: Pre-create volume

```
docker volume create paytm
docker run -it --name cont1 -v paytm:/data ubuntu
```

Method 4: In Dockerfile

```
FROM ubuntu
VOLUME ["/data"]
CMD ["bash"]
```

List all volumes:

```
docker volume ls
docker volume inspect paytm
```

7■■ REAL-WORLD EXAMPLES

Example 1: Development with Bind Mount

```
mkdir /root/myapp
cd /root/myapp
echo '<h1>Hello</h1>' > index.html

docker run -itd --name web \
  -v $(pwd):/usr/local/apache2/htdocs \
  -p 8080:80 httpd
```

Result: Edit index.html on host → Changes reflect in container instantly!

Example 2: Nginx with Named Volume

```
docker volume create web-data
docker run -itd --name nginx \
  -v web-data:/usr/share/nginx/html \
  -p 8080:80 nginx
```

Result: Persistent web content across container restarts

Example 3: PostgreSQL with Volume

```
docker volume create pgdata
docker run -itd --name postgres \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:15
```

Result: Database persists even if container dies

Example 4: Container-to-Container Sharing

```
# Container 1
docker run -itd --name app1 -v shared:/data ubuntu

# Container 2 (inherit volumes)
docker run -itd --name app2 --volumes-from app1 ubuntu

# Both access same /data volume!
```

Result: app2 automatically has access to all of app1's volumes

8■■ DOCKER COMPOSE + VOLUMES

```
version: '3.8'

services:
  postgres:
    image: postgres:15
    volumes:
      - pgdata:/var/lib/postgresql/data
    environment:
      POSTGRES_PASSWORD: secret

  app:
    image: myapp
    volumes:
      - app-logs:/app/logs
    depends_on:
      - postgres

volumes:
  pgdata:
    driver: local
  app-logs:
    driver: local
```

Advantages: Automatic volume management, multiple volumes, clean organization

9■■ BEST PRACTICES

■ **DO:** Use volumes for persistent data (databases, configs, uploads)

■ **DO:** Use bind mounts for development (edit code instantly)

■ **DO:** Name your volumes meaningfully (pgdata, web-content)

■ **DO:** Backup important volumes regularly

■ **DO:** Use docker-compose for managing multiple volumes

■ **DON'T:** Store data in container filesystem (it's lost on delete)

■ **DON'T:** Use --privileged=true unnecessarily

■ **DON'T:** Assume images contain volume data (they don't!)

QUICK REFERENCE COMMANDS

Task	Command
Create volume	`docker volume create mydata`
List volumes	`docker volume ls`
Inspect volume	`docker volume inspect mydata`
Run with named volume	`docker run -v mydata:/data ubuntu`
Run with bind mount	`docker run -v /root/app:/app ubuntu`
Run with --mount	`docker run --mount source=mydata,destination=/data ubuntu`
Inherit volumes	`docker run --volumes-from cont1 ubuntu`
Remove volume	`docker volume rm mydata`
Remove unused volumes	`docker volume prune`

■ **YOU NOW UNDERSTAND DOCKER VOLUMES!**
Ready to use in production. Practice with PostgreSQL, MongoDB, or any stateful service.